

● SECTION 1 — EMPLOYEE SQL PRACTICE

✚ TABLE: employees

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    salary DECIMAL(10,2),  
    hire_date DATE  
);
```

✚ INSERT DATA

```
INSERT INTO employees (emp_id, first_name, last_name, salary, hire_date) VALUES  
(101, 'Amit', 'Verma', 45000.00, '2020-03-15'),  
(102, 'Sneha', 'Patel', 62000.00, '2019-06-21'),  
(103, 'Rahul', 'Sharma', 82000.00, '2021-01-11'),  
(104, 'Priya', 'Reddy', 91000.00, '2023-09-10'),  
(105, 'Kunal', 'Nair', 75000.00, '2022-11-05'),  
(106, 'Anjali', 'Desai', 33000.00, '2020-02-03'),  
(107, 'Ravi', 'Kapoor', 67000.00, '2021-12-14'),  
(108, 'Meena', 'Bajaj', 88000.00, '2018-07-07'),  
(109, 'Arjun', 'Joshi', 52000.00, '2023-05-09'),  
(110, 'Divya', 'Iyer', 60000.00, '2022-04-27'),  
(111, 'Sanjay', 'Gupta', 43000.00, '2019-09-30'),  
(112, 'Neha', 'Malhotra', 71000.00, '2020-08-12'),  
(113, 'Vikram', 'Shetty', 99000.00, '2021-10-25'),  
(114, 'Pooja', 'Chopra', 58000.00, '2023-02-17'),
```

(115, 'Rohit', 'Mishra', 120000.00, '2024-01-05'),
(116, 'Isha', 'Rana', 79000.00, '2022-06-16'),
(117, 'Manish', 'Kumar', 35000.00, '2021-03-03'),
(118, 'Kavita', 'Singh', 93000.00, '2020-10-19'),
(119, 'Aditya', 'Saxena', 87000.00, '2019-05-08'),
(120, 'Nisha', 'Thakur', 65000.00, '2024-04-01'),
(121, 'Harsh', 'Dwivedi', 39000.00, '2018-08-14'),
(122, 'Ritika', 'Pandey', 34000.00, '2021-09-12'),
(123, 'Nitin', 'Chauhan', 72000.00, '2020-05-20'),
(124, 'Tanya', 'Bhatia', 85000.00, '2023-11-11'),
(125, 'Suresh', 'Gill', 30000.00, '2024-03-22'),
(126, 'Swati', 'Rajput', 69000.00, '2022-12-06'),
(127, 'Jay', 'Khatri', 97000.00, '2018-01-19'),
(128, 'Rani', 'Dubey', 36000.00, '2019-07-13'),
(129, 'Ankit', 'Yadav', 56000.00, '2020-06-29'),
(130, 'Sunita', 'Mehta', 47000.00, '2021-08-24'),
(131, 'Deepak', 'Tripathi', 81000.00, '2023-10-03'),
(132, 'Bhavna', 'Tiwari', 92000.00, '2022-02-10'),
(133, 'Raj', 'Aggarwal', 49000.00, '2024-07-15'),
(134, 'Komal', 'Saxena', 73000.00, '2020-12-31'),
(135, 'Aarav', 'Rastogi', 62000.00, '2018-03-28'),
(136, 'Poonam', 'Dhingra', 89000.00, '2019-10-10'),
(137, 'Siddharth', 'Chatterjee', 110000.00, '2023-06-05'),
(138, 'Lavanya', 'Pillai', 35000.00, '2021-04-17'),
(139, 'Raghav', 'Bhatt', 79000.00, '2022-09-09'),
(140, 'Namrata', 'Menon', 67000.00, '2020-01-01');

EMPLOYEE SQL QUESTIONS

(Students answer these)

1. Get average salary by hire year (highest → lowest).
 2. Employees hired per year (ascending).
 3. Top 3 hire years by total salary.
 4. Total salary per hire year (descending).
 5. Top 5 highest-paid employees.
 6. Average salary per hire year (ascending).
 7. Count employees earning > 70000 grouped by hire year (descending).
 8. Min and max salary per hire year.
 9. Full name as “Last Name, First Name”, sorted.
 10. First names starting with A.
 11. First names uppercase sorted reverse.
 12. First 3 letters of first_name sorted.
 13. Names with 'a' after position 2.
 14. Full name length > 15 sorted.
 15. Employees hired in December.
 16. Last names containing 'an'.
-

● SECTION 2 — TENNIS MATCH JOINS PRACTICE

📌 TABLE: matches

```
CREATE TABLE matches (  
    match_id INT PRIMARY KEY,  
    tournament VARCHAR(100),  
    match_date DATE,  
    round VARCHAR(20)  
);
```

📌 TABLE: match_players

```
CREATE TABLE match_players (  
    match_id INT,  
    player_a VARCHAR(100),  
    player_b VARCHAR(100),  
    winner VARCHAR(100),  
    FOREIGN KEY (match_id) REFERENCES matches(match_id)  
);
```

📌 TABLE: match_stats

```
CREATE TABLE match_stats (  
    match_id INT,  
    aces_player_a INT,  
    aces_player_b INT,  
    double_faults_a INT,  
    double_faults_b INT,  
    total_points_a INT,  
    total_points_b INT,
```

FOREIGN KEY (match_id) REFERENCES matches(match_id)

);

TENNIS SQL QUESTIONS

1. List all matches with tournament + players.
 2. All matches won by Rafael Nadal.
 3. Show aces for each match.
 4. Match with highest total aces.
 5. Matches where Player A had more double faults.
 6. Final round matches + winners.
 7. Show losing player using CASE.
 8. Average points by Player A.
 9. Total match points ordered by date.
-

● SECTION 3 — FOOD DELIVERY SYSTEM (CUSTOMERS / ORDERS)

📌 TABLE: customers

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(100)  
);
```

📌 TABLE: orders

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    order_date DATETIME,  
    total_amount DECIMAL(10,2),  
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

📌 TABLE: order_items

```
CREATE TABLE order_items (  
    item_id INT PRIMARY KEY,  
    order_id INT,  
    item_name VARCHAR(100),  
    quantity INT,  
    price DECIMAL(10,2),  
    FOREIGN KEY (order_id) REFERENCES orders(order_id)  
);
```

FOOD DELIVERY SQL QUESTIONS

1. Orders with customer names + dates.
 2. Items with customer name + order ID.
 3. Total amount spent by each customer.
 4. Customers with > 1 order.
 5. Items ordered more than once.
 6. Most recent order per customer.
 7. Total items in each order.
 8. Average order value per customer.
 9. Customers who ordered Pizza Margherita.
-

● SECTION 4 — ADVANCED JOINS (ECOMMERCE)

📌 TABLE: Customers

```
CREATE TABLE Customers (  
    CustomerID INT PRIMARY KEY,  
    CustomerName VARCHAR(100),  
    ContactName VARCHAR(100),  
    Country VARCHAR(50)  
);
```

📌 TABLE: Orders

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    OrderDate DATE,  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

📌 TABLE: Products

```
CREATE TABLE Products (  
    ProductID INT PRIMARY KEY,  
    ProductName VARCHAR(100),  
    Price DECIMAL(10, 2)  
);
```

📌 TABLE: OrderDetails

```
CREATE TABLE OrderDetails (  
    OrderID INT,  
    ProductID INT,
```

Quantity INT,
PRIMARY KEY (OrderID, ProductID),
FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);

ADVANCED SQL QUESTIONS

1. Total quantity per customer (only if >2).
 2. Total value per customer after 2025-09-01 (only >1 order).
 3. Products priced >50 total quantity per customer (only if >3 distinct).
 4. Products ordered >2 times + count distinct customers.
 5. Total value per customer in Sept 2025 (only >2 products).
-

● SECTION 5 — DATE FUNCTIONS (FLIGHTS / BOOKINGS)

✚ TABLE: Flights

```
CREATE TABLE Flights (  
    FlightID INT PRIMARY KEY,  
    FlightNumber VARCHAR(10),  
    DepartureDate DATETIME,  
    ArrivalDate DATETIME,  
    Origin VARCHAR(50),  
    Destination VARCHAR(50)  
);
```

✚ TABLE: Passengers

```
CREATE TABLE Passengers (  
    PassengerID INT PRIMARY KEY,  
    FirstName VARCHAR(50),  
    LastName VARCHAR(50),  
    DateOfBirth DATE  
);
```

✚ TABLE: Bookings

```
CREATE TABLE Bookings (  
    BookingID INT PRIMARY KEY,  
    FlightID INT,  
    PassengerID INT,  
    BookingDate DATETIME,  
    FOREIGN KEY (FlightID) REFERENCES Flights(FlightID),  
    FOREIGN KEY (PassengerID) REFERENCES Passengers(PassengerID)  
);
```

DATE FUNCTION QUESTIONS

1. Days between booking and departure.
 2. Year of departure.
 3. Flights in September 2025.
 4. Passengers who booked >10 days early.
 5. Bookings per month (2025).
 6. Average booking-to-departure days.
 7. Flight with most bookings.
 8. Flight with least bookings.
 9. Total revenue per flight (fare = 300).
 10. Average booking gap per origin city (>5).
 11. Passengers booking within 7 days (>1).
-

● SECTION 6 — AGGREGATIONS (CLOTHING SALES)

📌 TABLE: ClothingSales

```
CREATE TABLE ClothingSales (  
    SaleID INT PRIMARY KEY,  
    ProductName VARCHAR(100),  
    Category VARCHAR(50),  
    Price DECIMAL(10, 2),  
    Quantity INT,  
    SaleDate DATETIME  
);
```

📌 AGGREGATION QUESTIONS

1. Total revenue per category (2025).
2. Average price per month (2025).
3. Products with total quantity >10.
4. Footwear items with revenue >100.
5. Monthly revenue + highest month.
6. Top 3 products by quantity sold.
7. Average quantity per Apparel product.
8. Number of distinct products sold.
9. Revenue + quantity for Bottoms.
10. Revenue + quantity for products starting with “H”.